siga os passos

EAD UNIG UNIVERSIDADE IGUAÇU

GPAuni Ambiente Virtual de Aprendizagem

ROTEIRO

DE ATIVIDADES

pesquise

pratique

produza

Dados do projeto

Tipo de projeto: Não-Extensionista
Tema integrador: Tema Integrador VIII – Projeto de Linguagens
Período: 8º
Carga horária: 60h
Público-alvo da atividade:
Nome da atividade: Linguagem de Comunicação Alternativa e Aumentativa (CAA) e Testador em Python para Comunicação Acessível e Validação Formal

Objetivo da atividade

Esta atividade visa planejar, formalizar e validar uma Linguagem de Comunicação Alternativa e Aumentativa (CAA) por meio de BNF/EBNF, implementação de scanner e parser em Python (análise descendente recursiva), contemplando boas práticas de Engenharia de Linguagens, Testes e Acessibilidade Digital. O projeto promove a inclusão comunicacional ao permitir que enunciados essenciais ("Eu preciso…", "Eu quero…", "sim/não") sejam representados, reconhecidos e validados por software de forma confiável, verificável e reprodutível.

ODS principal: ODS 10 – Redução das Desigualdades

- Meta 10.2: promover a inclusão social de pessoas em situação de vulnerabilidade.
- Meta 10.3: garantir igualdade de oportunidades e reduzir desigualdades de resultados (incluindo remoção de barreiras comunicacionais).

Observação: o projeto também se co-alinha ao ODS 4 – Educação de Qualidade (metas 4.5 – eliminação de disparidades no acesso/participação; 4.a – ambientes de aprendizagem inclusivos e acessíveis), por fortalecer práticas pedagógicas inclusivas com tecnologia assistiva.

Indicadores dos Objetivos de Desenvolvimento Sustentável - Brasil



Atividades a serem desenvolvidas

Caro aluno(a),

Desenvolva as atividades descritas abaixo e insira as respostas no módulo de gestão de projetos, na fase projeto ("ATIVIDADES DO ROTEIRO"). Em caso de atividades individuais seguir com as instruções da mesma

forma, sendo que apenas um(a) aluno(a) será o responsável pelo trabalho. Verificar com o(a) professor(a) responsável as regras de composição do projeto.

Fiquem atentos para o feedback do(a) professor(a) após as atividades!

Bom projeto!

ESCOLHA UM PÚBLICO-ALVO

Estamos dando um exemplo de Linguagem, o aluno deve aumentar o corpus de linguagem ou pode também trabalhar outro público-alvo, deve ser validado com a(o) professora(or).

Atividades

Roteiro: Criando a Linguagem e o Testador em Python

Caro(a) aluno(a),

Para a execução das Atividades 1, 2 e 3 do seu projeto, você precisará definir a linguagem de Comunicação Alternativa e Aumentativa (CAA) e, em seguida, implementar o analisador léxico (scanner) e o analisador sintático (parser) em Python.

Aqui está um guia passo a passo de como proceder, unificando os conceitos e exemplos já apresentados.

Atividade 1.1: Embasamento para a pesquisa e realização da atividade.

Pesquisa do público-alvo para validação da linguagem (introdução e revisão de literatura)

Descrição:
Avalia a qualidade da pesquisa sobre o público-alvo escolhido (ex.: pessoas com afasia, TEA, TDL, idosos ou alunos da Educação Inclusiva). Considera-se a coerência entre a fundamentação teórica e a proposta da linguagem CAA, o uso de referências científicas atualizadas (artigos, teses, normas e diretrizes de comunicação alternativa), e a clareza na justificativa da escolha do vocabulário e das regras gramaticais frente às necessidades comunicacionais reais.

Indicadores de desempenho:

- Utilização de fontes científicas verificáveis (SciELO, PubMed, Capes, etc.);
- Articulação entre a teoria e a prática no contexto da acessibilidade comunicacional;
- Apresentação de dados, autores e conceitos-chave sobre Comunicação Alternativa e Aumentativa;
- Clareza na delimitação do público-alvo e na aplicabilidade social da linguagem desenvolvida.

Atividade 1.2: Definindo a Linguagem de CAA (Gramática)

Nesta etapa, você usará a **Notação BNF (Backus-Naur Form)** ou EBNF para **formalizar as regras de como as frases válidas** podem ser construídas usando seus símbolos/palavras.

1. Escolha seu vocabulário (Símbolos/Palavras):

- `Eu`, `Você` (Sujeitos)
- `quero`, `preciso` (Verbos Auxiliares)
- `água`, `comida` (Objetos)
- `sair`, `comer` (Verbos no Infinitivo)
- `sim`, `não` (Confirmação/Negação)

2. Defina as regras gramaticais em BNF:

Use um editor de texto simples para escrever suas regras.

```
# Regras Principais BNF
<frase> ::= <resposta_simples> | <declaracao>

# Regra para frases como "sim" ou "não"
<resposta_simples> ::= <CONFIRMACAO> | <NEGACAO>

# Regra para frases como "Eu quero água" ou "Eu quero sair"
<declaracao> ::= <SUJEITO> <acao>

# Regra para definir o que é uma ação (pode ser um verbo auxiliar + infinitivo OU verbo auxiliar + objeto)
<acao> ::= <VERBO_AUX> <VERBO_INF>
        | <VERBO_AUX> <OBJETO>
```

```
# Definições dos Tokens (Terminais)
<SUJEITO> ::= "Eu" | "Você"
<VERBO_AUX> ::= "quero" | "preciso"
<OBJETO> ::= "agua" | "comida"
<VERBO_INF> ::= "sair" | "comer"
<CONFIRMACAO> ::= "s" | "sim"
<NEGACAO> ::= "n" | "nao"
```

Atividade 2: Criando o Analisador Léxico (Scanner) em Python

Agora, você usará a biblioteca `re` do Python para criar o scanner que transforma o texto de entrada (sequência de símbolos) em uma lista de *tokens*, seguindo as definições da Atividade 1.

Código Python do Scanner (`scanner.py`):

```python
import re

# Mapeamento dos tipos de tokens para suas expressões regulares
TOKEN_TYPES = [
    # Ordem de cima para baixo é importante!
    ('SUJEITO', r'Eu|Voce'),
    ('VERBO_AUX', r'quero|preciso'),
    ('OBJETO', r'agua|comida'),
    ('VERBO_INF', r'sair|comer|sa'), # Incluindo 'sa' como símbolo de sair
    ('CONFIRMACAO', r's|sim'),
    ('NEGACAO', r'n|nao'),
    ('SKIP', r'[ \t\n]+'),              # Ignorar espaços e quebras de linha
    ('MISMATCH', r'.'),                 # Capturar qualquer outra coisa como erro
]

token_regex = '|'.join('(?P<%s>%s)' % (name, pattern) for name, pattern in TOKEN_TYPES)

def lexical_analyzer(source_code):
    tokens = []
```

```
    for match in re.finditer(token_regex, source_code):
        token_type = match.lastgroup
        token_value = match.group(token_type)

        if token_type == 'SKIP':
            continue

        if token_type == 'MISMATCH':
            raise RuntimeError(f"Erro léxico: Caractere/simbolo inesperado '{token_value}' na posição {match.start()}")

        tokens.append((token_type, token_value))
    return tokens

# Exemplo de teste:
# input_frase = "Eu quero sa"
# tokens = lexical_analyzer(input_frase)
# print(tokens)
# Saída: [('SUJEITO', 'Eu'), ('VERBO_AUX', 'quero'), ('VERBO_INF', 'sa')]
```

Atividade 3: Criando o Analisador Sintático (Parser) em Python

Aqui, você implementa a lógica das regras BNF usando funções Python (Análise Descendente Recursiva). O parser usará os *tokens* do scanner para validar a estrutura da frase.

Código Python do Parser (`parser.py`):

```
# Importe a função do seu scanner, ou cole o código acima neste arquivo
# from scanner import lexical_analyzer

class Parser:
    def __init__(self, tokens):
        self.tokens = tokens
        self.current_token_index = 0
```

```
        self.current_token_type = tokens[0][0] if tokens else None
        self.current_token_value = tokens[0][1] if tokens else None

    def advance(self):
        """Move para o próximo token."""
        self.current_token_index += 1
        if self.current_token_index < len(self.tokens):
            self.current_token_type = self.tokens[self.current_token_index][0]
            self.current_token_value = self.tokens[self.current_token_index][1]
        else:
            self.current_token_type = None
            self.current_token_value = None

    def consume(self, expected_type):
        """Verifica e consome o token esperado, ou lança erro."""
        if self.current_token_type == expected_type:
            print(f"Consumido: ({expected_type}, '{self.current_token_value}')")
            self.advance()
        else:
            raise SyntaxError(f"Erro de sintaxe: Esperado '{expected_type}', mas encontrado '{self.current_token_type}'
('{self.current_token_value}')")

    # Implementação das Regras BNF como métodos Python

    def parse_frase(self):
        """<frase> ::= <resposta_simples> | <declaracao>"""
        if self.current_token_type in ['CONFIRMACAO', 'NEGACAO']:
            self.parse_resposta_simples()
        elif self.current_token_type == 'SUJEITO':
            self.parse_declaracao()
        else:
            raise SyntaxError("Erro: Frase não inicia com confirmação ou sujeito válido.")
```

```
        print("\nAnálise sintática concluída: Frase válida.")

    def parse_resposta_simples(self):
        """<resposta_simples> ::= <CONFIRMACAO> | <NEGACAO>"""
        if self.current_token_type == 'CONFIRMACAO':
            self.consume('CONFIRMACAO')
        elif self.current_token_type == 'NEGACAO':
            self.consume('NEGACAO')

    def parse_declaracao(self):
        """<declaracao> ::= <SUJEITO> <acao>"""
        self.consume('SUJEITO')
        self.parse_acao()

    def parse_acao(self):
        """<acao> ::= <VERBO_AUX> <VERBO_INF> | <VERBO_AUX> <OBJETO>"""
        if self.current_token_type == 'VERBO_AUX':
            self.consume('VERBO_AUX')
            # Verifica se o próximo é INF (quero sair) ou OBJ (quero água)
            if self.current_token_type == 'VERBO_INF':
                self.consume('VERBO_INF')
            elif self.current_token_type == 'OBJETO':
                self.consume('OBJETO')
            else:
                raise SyntaxError("Erro: Esperado verbo infinitivo ou objeto após o verbo auxiliar.")
        else:
            raise SyntaxError("Erro: Esperado um verbo auxiliar.")


# --- Testador da Linguagem (Exemplo de Uso) ---

def testar_frase(texto_entrada):
```

```
    print(f"\n--- Testando: '{texto_entrada}' ---")
    try:
        tokens = lexical_analyzer(texto_entrada)
        print(f"Tokens: {tokens}")
        parser = Parser(tokens)
        parser.parse_frase()
    except (RuntimeError, SyntaxError) as e:
        print(f"Resultado da análise: {e}")


# Testes
testar_frase("Eu quero sa")     # Válido (Eu quero sair)
testar_frase("Eu quero agua")   # Válido (Eu quero água)
testar_frase("sim")             # Válido (Sim)
testar_frase("n")               # Válido (Não)
testar_frase("Eu agua quero")   # Inválido (Ordem errada)
```

Cronograma do projeto

Caro aluno(a),

Fique atento ao prazo para entrega das atividades do projeto. Esses prazos são importantes para obtenção do feedback contínuo do professor, que auxiliarão na composição da sua nota parcial.

Para obtenção e feedback parcial é necessária a inclusão da etapa ANTES da data final de entrega.

Etapa do projeto no sistema	Descrição	Data de Entrega
Start	Apresentação do projeto, regras para composição de grupos (quando existir a possibilidade de trabalho individual o professor irá informar) e definição do tema. Orientação sobre o Roteiro.	Aula síncrona agendada pelo docente. Fique atento(a)!
Atividades do Roteiro	Utilize o campo do sistema para realizar/responder as atividades deste roteiro.	Prazo Final de Envio Veja no Ambiente Virtual
Relatório da Atividade (Nota Parcial)	Preencher os campos seguindo as etapas do relatório.	Prazo Final de Envio Veja no Ambiente Virtual

Vídeo de Apresentação **(Nota Final)**	Apresentação final do projeto, insira o link do vídeo de apresentação conforme instruções no ambiente virtual(máx. 5 min)	Veja no Ambiente Virtual

Instruções para o vídeo de apresentação do projeto (Etapa final)

O vídeo deve realizado pelo grupo, **contendo todos os componentes em apresentação ou individual conforme orientação**. Deve explicar em no máximo 5 minutos, usando o Modelo de Apresentação da UNIG-EAD, a execução de seu trabalho e seus resultados alcançados.

Deve ser gerado um link deste vídeo, conforme vídeo explicativo que se encontra na etapa do vídeo de apresentação no Ambiente Virtual.

Avaliação do projeto

[Professor – Caso haja necessidade adicione critérios de avaliação e redistribua a pontuação]

Avaliação parcial (Roteiro e relatório) – 7,0pts	
Critério de Avaliação	**Peso**
Pesquisa do público-alvo para validação da linguagem (introdução e revisão de literatura).	2 pts
Funcionamento do testador	2 pts
Estruturação do planejamento e viabilidade da proposta de acordo com as instruções.	2 pts
Apresentação e organização do relatório	1 pt

Avaliação final (vídeo de apresentação) – 3,0pts	
Critério de Avaliação	**Peso**
Qualidade da entrega final (organização e cumprimento do roteiro da apresentação)	1 pts
Implementação das melhorias e correções	1 pts
Clareza e objetividade da apresentação em vídeo	1 pts

Considerações finais

- ✓ O projeto deve seguir as diretrizes estabelecidas.
- ✓ Os arquivos (se existirem) enviados devem ser apenas em ÚNICO ENVIO por grupo ou individual se permitido pela orientação.
- ✓ O uso de referências e citações será valorizado.
- ✓ As datas/prazos devem ser respeitadas.

Ouça nosso podcast sobre dicas para trabalho em equipe!



Aponte a câmera para o QrCode para ouvir o Podcast

✓ O vídeo deve ser claro e demonstrar domínio do conteúdo. Todos devem aparecer no vídeo.

✓ Os alunos(as) devem ficar atentos ao feedback do professor.

Esse roteiro contém a estrutura do Tema Integrador e é vinculado a sua prática, ambiente para gerir os Projetos Integradores e outros.

Para auxiliar

Momentos Síncronos, Tutoriais e Manual do Aluno

É importante sempre comparecer aos momentos síncronos, fazer a leitura do Manual do Aluno e assistir as tutoriais disponíveis. Em caso de dúvidas busque ajuda do(a) professor(a)!